

INTERFACE

Fullscreen rendering

Enable a smoother, flicker-free rendering mode with mouse support and stable memory usage in long conversations.

ⓘ Fullscreen rendering is an opt-in [research preview](#) and requires Claude Code v2.1.89 or later. Run `/tui fullscreen` to switch in your current conversation, or set `CLAUDE_CODE_NO_FLICKER=1` on versions before v2.1.110. Behavior may change based on feedback.

Fullscreen rendering is an alternative rendering path for the Claude Code CLI that eliminates flicker, keeps memory usage flat in long conversations, and adds mouse support. It draws the interface on the terminal's alternate screen buffer, like `vim` or `htop`, and only renders messages that are currently visible. This reduces the amount of data sent to your terminal on each update.

The difference is most noticeable in terminal emulators where rendering throughput is the bottleneck, such as the VS Code integrated terminal, tmux, and iTerm2. If your terminal scroll position jumps to the top while Claude is working, or the screen flashes as tool output streams in, this mode addresses those.

ⓘ The term fullscreen describes how Claude Code takes over the terminal's drawing surface, the way `vim` does. It has nothing to do with maximizing your terminal window, and works at any window size.

Enable fullscreen rendering

Run `/tui fullscreen` inside any Claude Code conversation. The CLI saves the [tui setting](#) and relaunches into fullscreen with your conversation intact, so you can switch mid-session without losing context. Run `/tui default` to switch back to the classic renderer, or `/tui` with no argument to print which renderer is active.

You can also set the `CLAUDE_CODE_NO_FLICKER` environment variable before starting Claude Code:

```
CLAUDE_CODE_NO_FLICKER=1 claude
```

The `tui` setting and the environment variable are equivalent. The `/tui` command clears `CLAUDE_CODE_NO_FLICKER` from the relaunched process so the setting it writes takes effect.

What changes

Fullscreen rendering changes how the CLI draws to your terminal. The input box stays fixed at the bottom of the screen instead of moving as output streams in. If the input doesn't move while Claude is working, fullscreen rendering is active. Only visible messages are kept in the render tree, so memory stays constant regardless of conversation length.

Because the conversation lives in the alternate screen buffer instead of your terminal's scrollback, a few things work differently:

Before	Now	Details
<code>Cmd+f</code> or tmux search to find text	<code>Ctrl+o</code> for transcript mode, then <code>/</code> to search or <code>[</code> to write to scrollback	<u>Search and review the conversation</u>
Terminal's native click-and-drag to select and copy	In-app selection, copies automatically on mouse release	<u>Use the mouse</u>
<code>Cmd</code> -click to open a URL	<code>Cmd</code> -click on macOS, <code>Ctrl</code> -click elsewhere	<u>Use the mouse</u>

If mouse capture interferes with your workflow, you can turn it off while keeping the flicker-free rendering.

Use the mouse

Fullscreen rendering captures mouse events and handles them inside Claude Code:

- **Click in the prompt input** to position your cursor anywhere in the text you're typing.
- **Click a suggestion in the `/` command or `@` file list** to accept it. Hovering highlights the row under your cursor.
- **Click an option in a select menu** to choose it. This covers permission prompts, `/model`, `/config`, and other dialogs that show a list of options. Hovering shows a pointer on the row under your cursor. Requires Claude Code v2.1.187 or later.
- **Click a collapsed tool result** to expand it and see the full output. Click again to collapse. The tool call and its result expand together. Only messages that have more to show are clickable.
- **Hold `Cmd` on macOS, or `Ctrl` on Linux and Windows, and click a URL or file path** to open it.

File paths in tool output, like the ones printed after an Edit or Write, open in your default application. Plain `http://` and `https://` URLs open in your browser. As of v2.1.181, a plain click without holding `Cmd` or `Ctrl` no longer opens links, matching native terminal behavior. In the VS Code integrated terminal and similar xterm.js-based terminals, Claude Code defers to the terminal's own link handler, which uses the same gesture.

- **Click and drag** to select text anywhere in the conversation. Double-click selects a word, matching iTerm2's word boundaries so a file path selects as one unit. Triple-click selects the line.
- **Scroll with the mouse wheel** to move through the conversation.

Selected text copies to your clipboard automatically on mouse release. To turn this off, toggle Copy on select in `/config`.

With Copy on select off, press `Ctrl+Shift+c` to copy manually. On terminals that support the kitty keyboard protocol, such as kitty, WezTerm, Ghostty, and iTerm2, `Cmd+c` also works. If you have a selection active, `Ctrl+c` copies instead of cancelling.

With a selection active, hold `Shift` and press the arrow keys to extend it from the keyboard. `Shift+↑` and `Shift+↓` scroll the viewport when the selection reaches the top or bottom edge. `Shift+Home` and `Shift+End` extend to the start or end of the current line.

Scroll the conversation

Fullscreen rendering handles scrolling inside the app. Use these shortcuts to navigate:

Shortcut	Action
<code>PgUp</code> / <code>PgDn</code>	Scroll up or down by half a screen
<code>Ctrl+Home</code>	Jump to the start of the conversation
<code>Ctrl+End</code>	Jump to the latest message and re-enable auto-follow
Mouse wheel	Scroll a few lines at a time

On keyboards without dedicated `PgUp`, `PgDn`, `Home`, or `End` keys, like MacBook keyboards, hold `Fn` with the arrow keys: `Fn+↑` sends `PgUp`, `Fn+↓` sends `PgDn`, `Fn+←` sends `Home`, and `Fn+→` sends `End`. That makes `Ctrl+Fn+→` the jump-to-bottom shortcut. If that feels awkward, scroll to the bottom with the mouse wheel to resume following, or rebind `scroll:bottom` to something reachable.

These actions are rebindable. See [Scroll actions](#) for the full list of action names, including half-page

and full-page variants that have no default binding.

Auto-follow

Scrolling up pauses auto-follow so new output doesn't pull you back to the bottom. Press `Ctrl+End` or scroll to the bottom to resume following.

To turn auto-follow off entirely so the view stays where you leave it, open `/config` and set Auto-scroll to off. With auto-scroll disabled, the view never jumps to the bottom on its own. Permission prompts and other dialogs that need a response still scroll into view regardless of this setting.

Mouse wheel scrolling

Mouse wheel scrolling requires your terminal to forward mouse events to Claude Code. Most terminals do this whenever an application requests it. iTerm2 makes it a per-profile setting: if the wheel does nothing but `PgUp` and `PgDn` work, open Settings → Profiles → Terminal and turn on Enable mouse reporting. The same setting is also required for click-to-expand and text selection to work.

If mouse wheel scrolling feels slow, your terminal may be sending one scroll event per physical notch with no multiplier. Some terminals, like Ghostty and iTerm2 with faster scrolling enabled, already amplify wheel events. Others, including the VS Code integrated terminal, send exactly one event per notch. Claude Code can't detect which.

Set `CLAUDE_CODE_SCROLL_SPEED` to multiply the base scroll distance:

```
export CLAUDE_CODE_SCROLL_SPEED=3
```

A value of `3` matches the default in `vim` and similar applications. The setting accepts values from 1 to 20, and fractional values below 1 such as `0.5` to slow accelerated trackpad and wheel scrolling in terminals that already amplify wheel events.

To adjust scroll speed interactively, run `/scroll-speed`. The dialog shows a ruler you can scroll while it is open so you can feel the change immediately. Press `←` and `→` to adjust, `r` to reset to the auto-detected default, and `Enter` to save.

The command writes the same value the `CLAUDE_CODE_SCROLL_SPEED` environment variable sets, persisted to `~/.claude/settings.json`. The command isn't available in the JetBrains IDE terminal.

Separately from the base speed, Claude Code accelerates the scroll rate when you spin the wheel quickly, so a fast spin covers more distance than the same number of slow notches. To turn acceleration off and keep a constant rate per notch, set `wheelScrollAccelerationEnabled` to `false` in `settings.json`. This setting requires Claude Code v2.1.174 or later.

Scroll in the JetBrains IDE terminal

In the JetBrains IDE terminal, Claude Code applies its own scroll handling and ignores `CLAUDE_CODE_SCROLL_SPEED`. The terminal sends scroll events at a much higher rate than other emulators, so a multiplier tuned elsewhere overshoots here.

In 2025.2, the terminal also has scroll-wheel bugs that produce spurious arrow keys and wrong-direction events. Claude Code detects these at runtime and mitigates them automatically, so trackpad and mouse wheel scrolling work without configuration. For the best scroll experience, upgrade to 2025.3 or later. Claude Code shows a hint the first time you scroll if it detects the bug.

Search and review the conversation

`Ctrl+o` toggles between the normal prompt and transcript mode.

For a quieter view that shows only your last prompt, a one-line summary of tool calls with edit diffstats, and the final response, run `/focus`. The setting persists across sessions. Run `/focus` again to turn it off.

Transcript mode gains `less`-style navigation and search:

Key	Action
<code>/</code>	Open search. Type to find matches, <code>Enter</code> to accept, <code>Esc</code> to cancel and restore your scroll position
<code>n</code> / <code>N</code>	Jump to next or previous match. Works after you've closed the search bar
<code>j</code> / <code>k</code> or <code>↑</code> / <code>↓</code>	Scroll one line
<code>g</code> / <code>G</code> or <code>Home</code> / <code>End</code>	Jump to top or bottom
<code>Ctrl+u</code> / <code>Ctrl+d</code>	Scroll half a page
<code>Ctrl+b</code> / <code>Ctrl+f</code> or <code>Space</code> / <code>b</code>	Scroll a full page
<code>Ctrl+o</code> , <code>Esc</code> , or <code>q</code>	Exit transcript mode and return to the prompt

Your terminal's `Cmd+f` and tmux search don't see the conversation because it lives in the alternate screen buffer, not the native scrollback. To hand the content back to your terminal, press `Ctrl+o` to enter transcript mode first, then:

- `[` : writes the full conversation into your terminal's native scrollback buffer, with all tool output expanded. The conversation is now ordinary text in your terminal, so `Cmd+f`, tmux copy mode, and any other native tool can search or select it. Long sessions may pause for a moment while this happens. This lasts until you exit transcript mode with `Esc` or `q`, which returns you to fullscreen rendering. The next `Ctrl+o` starts fresh.
- `v` : writes the conversation to a temporary file and opens it in `$VISUAL` or `$EDITOR`.

Press `Esc` or `q` to return to the prompt.

Clear the conversation

Press `Ctrl+L` twice within two seconds to run `/clear` and start a new conversation. The first press redraws the screen and shows a hint; the second press clears the conversation. On macOS, double-pressing `Cmd+K` also runs `/clear`.

Use with tmux

Fullscreen rendering works inside tmux, with three caveats.

Mouse wheel scrolling requires tmux's mouse mode. If your `~/.tmux.conf` doesn't already enable it, add this line and reload your config:

```
set -g mouse on
```

Without mouse mode, wheel events go to tmux instead of Claude Code. Keyboard scrolling with `PgUp` and `PgDn` works either way. Claude Code prints a one-time hint at startup if it detects tmux with mouse mode off.

Fullscreen rendering is incompatible with iTerm2's tmux integration mode, which is the mode you enter with `tmux -CC`. In integration mode, iTerm2 renders each tmux pane as a native split rather than letting tmux draw to the terminal. The alternate screen buffer and mouse tracking don't work correctly there: the mouse wheel does nothing, and double-click can corrupt the terminal state. Don't enable fullscreen rendering in `tmux -CC` sessions. Regular tmux inside iTerm2, without `-CC`, works fine.

tmux doesn't support synchronized output, so you may see more flicker during redraws than when running Claude Code directly in your terminal. If the flicker is noticeable, especially over SSH, run Claude Code in its own terminal tab outside tmux.

Keep native text selection

Mouse capture is the most common friction point, especially over SSH or inside tmux. When Claude Code captures mouse events, your terminal's native copy-on-select stops working. The selection you make with click-and-drag exists inside Claude Code, not in your terminal's selection buffer, so tmux copy mode, Kitty hints, and similar tools don't see it.

Claude Code writes the selection to your system clipboard, and the path it uses depends on your setup. On a local session it runs a native clipboard tool:

- **macOS:** `pbcopy`
- **Linux:** `wl-copy` on Wayland, or `xclip` or `xsel` on X11, whichever is installed. Claude Code writes both the clipboard and the PRIMARY selection, so middle-click paste works.
- **Windows and WSL:** PowerShell `Set-Clipboard`

Inside tmux it also writes to the tmux paste buffer. Over SSH it falls back to OSC 52 escape sequences. Claude Code prints a toast after each copy telling you which path it used.

Some terminals block OSC 52 by default. iTerm2 blocks it until you turn on Settings → General → Selection → Applications in terminal may access clipboard; running `/terminal-setup` in iTerm2 enables this for you.

For a one-off native selection, the key to use depends on your terminal:

- **Terminal.app:** `Fn`
- **iTerm2:** `Option`
- **VS Code, Cursor, and Devin Desktop:** `Shift`, or `Option` on macOS with the `terminal.integrated.macOptionClickForcesSelection` setting enabled
- **Most other terminals:** `Shift`

Hold that key while you click and drag. Your terminal handles the selection itself instead of passing it to Claude Code, so copy shortcuts like `Cmd+C` work on what you select. Claude Code also shows the correct key in its on-screen hint.

Over SSH or inside tmux, Claude Code can't always detect the terminal you're connecting from, so

the hint lists the candidate keys instead.

If you rely on native selection all the time, set `CLAUDE_CODE_DISABLE_MOUSE=1` to opt out of mouse capture while keeping the flicker-free rendering and flat memory:

```
CLAUDE_CODE_NO_FLICKER=1 CLAUDE_CODE_DISABLE_MOUSE=1 claude
```

With mouse capture disabled, keyboard scrolling with `PgUp`, `PgDn`, `Ctrl+Home`, and `Ctrl+End` still works, and your terminal handles selection natively. You lose click-to-position-cursor, click-to-expand tool output, URL clicking, and wheel scrolling inside Claude Code.

Research preview

Fullscreen rendering is a research preview feature. It has been tested on common terminal emulators, but you may encounter rendering issues on less common terminals or unusual configurations.

If you encounter a problem, run `/feedback` inside Claude Code to report it, or open an issue on the [claude-code GitHub repo](#). Include your terminal emulator name and version.

To turn fullscreen rendering off, run `/tui default`, or unset `CLAUDE_CODE_NO_FLICKER` if you enabled it that way. To force the classic renderer regardless of the saved `tui` setting, set `CLAUDE_CODE_DISABLE_ALTERNATE_SCREEN=1`. The classic renderer keeps the conversation in your terminal's native scrollback so `Cmd+f` and tmux copy mode work as usual.

Background sessions opened from [agent view](#) or `claude attach` always use fullscreen rendering. The attaching terminal enters the alternate screen buffer to show the session, and the classic renderer has no scrollback or mouse handling there, so the `tui` setting and `CLAUDE_CODE_DISABLE_ALTERNATE_SCREEN` don't apply to them.